

Aula 06 — Structured Output e Tool Use

SGAIM 2025/2026 — Semana 6

Contexto

Na semana 5 explorámos o LLM como motor isolado — stateless, sem ferramentas, sem memória. Hoje resolvemos uma dessas limitações: damos-lhe **mãos**. O LLM passa de “pensar” a “agir”.

Vão explorar um mini agent stack em Python que mostra a progressão completa: texto livre → JSON mode → structured output → tool calling. Primeiro com um mock determinístico (para compreender a arquitectura), depois com um modelo real via Ollama (para confrontar a realidade).

Método: antes de cada desafio, escrevam numa frase o que esperam que aconteça. Depois testem. Confirmou-se?

Pré-requisitos

Software

- Python 3.10+ com pydantic>=2.6 e requests>=2.31
- Ollama funcional (já instalado na aula 05) com pelo menos llama3.2:3b

O package

Descarreguem o mini_agent_stack/ do Moodle e instalem as dependências:

```
cd mini_agent_stack
python -m venv .venv
```

```
# Windows
.venv\Scripts\activate
```

```
# macOS/Linux
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Verifiquem que funciona:

```
python demo.py
```

Devem ver 3 exemplos a correr (weather, cálculo, pergunta geral), cada um com vários estágios de output.

Estrutura do package

```
mini_agent_stack/
├── schemas.py      ← Pydantic: JsonModeIntent, AgentDecision, WeatherArgs, ...
├── tools.py        ← 2 ferramentas: get_weather (fake), calculator (safe)
├── llm_clients.py  ← MockLLMClient (determinístico) + OllamaCompatibleClient (real)
├── controller.py   ← MiniAgentController: valida output + executa tools
└── demo.py         ← 3 exemplos: weather, cálculo, pergunta geral
```

Se não tiverem Ollama funcional, trabalhem em par com um colega que tenha.

Parte 1 — Explorar o stack com mock

Trabalho em pares. Anotem observações — vão partilhar com a turma.

O `MockLLMClient` é determinístico: mesma pergunta, mesma resposta. Isto é intencional — permite compreender a arquitectura sem variabilidade do modelo.

Desafio 1 — Correr o demo (~5 min)

Corram `python demo.py` e observem o output. Para cada um dos 3 inputs, o demo mostra vários estágios:

1. **JSON mode** — o modelo emite JSON com `intent` e `confidence`
2. **Structured decision** — o modelo decide: responder directamente ou chamar ferramenta
3. **Tool result** (se aplicável) — o sistema executa a ferramenta
4. **Final answer** — a resposta ao utilizador

Perguntas: - Qual dos 3 exemplos segue o caminho da ferramenta? Qual responde directamente? - Que informação se ganha em cada estágio? - O que aconteceria se o estágio 1 (JSON mode) fosse o único? Seria suficiente para executar acções?

Desafio 2 — Provocar um `ValidationError` (~10 min)

Abram `llm_clients.py` e encontrem o método `MockLLMClient.structured_decision()`.

Tarefa: editem temporariamente o mock para devolver JSON inválido segundo o schema. Exemplos para tentar:

- Devolver `{"action": "call_tool"}` sem `tool_name` — o que diz o Pydantic?
- Devolver `{"action": "call_tool", "tool_name": "get_weather"}` sem `arguments`
- Devolver `{"action": "respond"}` sem `answer`
- Devolver `{"action": "fly_to_moon"}` — um action que não existe no schema

Corram `python demo.py` após cada alteração.

Perguntas: - A mensagem de erro do Pydantic é clara? Ajuda a diagnosticar o problema? - Preferem que o sistema falhe ruidosamente aqui, ou silenciosamente a jusante? - Quem faz esta validação — o modelo ou o sistema?

Não se esqueçam de repor o mock original antes de avançar.

Desafio 3 — Desenhar o fluxo (~10 min)

Com papel e caneta, leiam `controller.py` e tracem o fluxo de dados do método `run_full_loop()`:

1. De onde vem o input?
2. Que métodos são chamados e em que ordem?
3. Onde é que o fluxo bifurca (`respond` vs `call_tool`)?
4. Quantas vezes o LLM é chamado no caminho da ferramenta? E no caminho directo?

Comparem o vosso diagrama com o diagrama dos slides. Há diferenças? O código tem mais passos do que o diagrama simplificado?

Desafio 4 — Adicionar uma 3.ª ferramenta (~15 min)

O stack tem 2 ferramentas: `get_weather` e `calculator`. Adicionem uma terceira.

Sugestões (escolham uma ou inventem): - `translate(text, target_language)` — tradução (pode ser fake) - `schedule_meeting(person, date, time)` — marcação de reunião (pode ser fake) - `unit_convert(value, from_unit, to_unit)` — conversão de unidades

O que precisam de mudar:

Ficheiro	O que fazer
<code>schemas.py</code>	1. Criar um <code>BaseModel</code> para os argumentos (ex: <code>TranslateArgs</code>)
	2. Adicionar o nome ao <code>Literal</code> de <code>tool_name</code> em <code>AgentDecision</code>
	3. Adicionar à <code>TOOL_ARG_SCHEMAS</code>
<code>tools.py</code>	4. Implementar a função (pode ser fake, ex: <code>return {"translated": f"[{lang}]{text}"}</code>)
	5. Adicionar à <code>TOOLS</code>
<code>llm_clients.py</code>	6. Actualizar <code>MockLLMClient.structured_decision()</code> para reconhecer pedidos da nova ferramenta
<code>controller.py</code>	7. Actualizar a <code>TOOLS_SPEC</code> com a descrição da nova ferramenta

Teste: corram `demo.py` com um input que active a nova ferramenta. Todo o fluxo passa?

Parte 2 — Com modelo real (Ollama)

Continuação em pares. O objectivo é confrontar a arquitectura perfeita com a realidade imperfeita de um modelo real.

Desafio 5 — Trocar mock por Ollama (~5 min)

No `demo.py`, mudem a linha:

```
controller = MiniAgentController(MockLLMClient())
```

para:

```
from llm_clients import OllamaCompatibleClient
controller = MiniAgentController(
    OllamaCompatibleClient("http://localhost:11434", "llama3.2:3b")
)
```

Corram os **mesmos 3 exemplos**. O resultado vai ser diferente — e provavelmente imperfeito.

Pergunta: Antes de correr — o que acham que vai acontecer com a pergunta "Explain what a transformer is"? Tool call ou resposta directa?

Desafio 6 — Registar falhas (~15 min)

Corram vários inputs e documentem **cada falha** do modelo real. Classifiquem-nas:

Tipo de falha	Exemplo	A validação apanha?
JSON inválido (não parseable)	Sure! Here's the weather...	Sim — <code>json.loads</code> falha
JSON válido, schema inválido	<code>{"action": "call_tool"}</code> sem <code>tool_name</code>	Sim — Pydantic rejeita
Ferramenta errada	Escolhe calculator para weather	Não — schema válido
Argumentos inventados	<code>{"city": "Lisboa", "forecast_type": "extended"}</code>	Não — Dict aceita tudo
Responde quando devia chamar	Resposta directa a "weather in Lisbon"	Não — schema válido

Pergunta-chave: Qual destas falhas seria apanhada pela validação? Qual passaria silenciosamente? O que é que isto vos diz sobre os limites da validação por schema?

Desafio 7 — Melhorar prompts (~10 min)

Abram `llm_clients.py` e editem os system prompts do `OllamaCompatibleClient`. Experimentem:

- Adicionar o schema JSON completo ao system prompt (com exemplos dos campos)
- Usar **few-shot**: incluir 2 exemplos de tool calls correctas no prompt
- Ser mais assertivo: "You MUST respond with ONLY valid JSON. No markdown, no explanation."
- Experimentar `temperature: 0` vs `temperature: 0.3`

O que registar: Que mudança fez mais diferença? Porquê?

Desafio 8 — Comparar modelos (~10 min)

Testem com modelos de tamanhos diferentes:

```
OllamaCompatibleClient("http://localhost:11434", "llama3.2:1b")
OllamaCompatibleClient("http://localhost:11434", "llama3.2:3b")
OllamaCompatibleClient("http://localhost:11434", "qwen2.5:7b") # se tiverem RAM
```

Para cada modelo, registem: - Taxa de JSON válido (em 5 tentativas) - Taxa de ferramenta correcta - Qualidade dos argumentos

Pergunta: Qual é o modelo mais pequeno que acerta consistentemente em tool calling?

Para terminar

No final da aula vamos partilhar descobertas. Preparem:

- Qual foi a falha mais interessante que o modelo real cometeu?
- Que mudança no prompt fez mais diferença?
- O mock é “irrealista” — mas qual é a sua vantagem para compreender a arquitetura?

TPC: - Completar/refinar o `OllamaCompatibleClient` com melhores prompts - Ter pelo menos 1 ferramenta adicional implementada no stack - Desafio extra: que acontece se o schema tiver 5+ ferramentas? O modelo continua fiável?

Na semana 7 começamos RAG — o cérebro ganha memória. Começa também o **mini-project 2** (RAG pipeline).